NATIONAL UNIVERSITY
OF COMPUTER & EMERGING SCIENCES PESHAWAR CAMPUS

Name:	....	**Roll No:**	....
Program:	....	**Examination:**	Sessional-II
Semester:	Fall 2019	**Total marks:** 20	**Weight:** x
Time allowed:	60 mins	**Date:**	October, 2019
Course:	CS-218 Data Structures	**Instructor:**	Nauman

Q. No.:	1	2	3	4	5	6	Sum	Sign
:								
Total:	3	3	4	4	3	3		

Attempt all questions on the question sheet. Answer the questions as concisely as possible. Keep your text within the provided space.

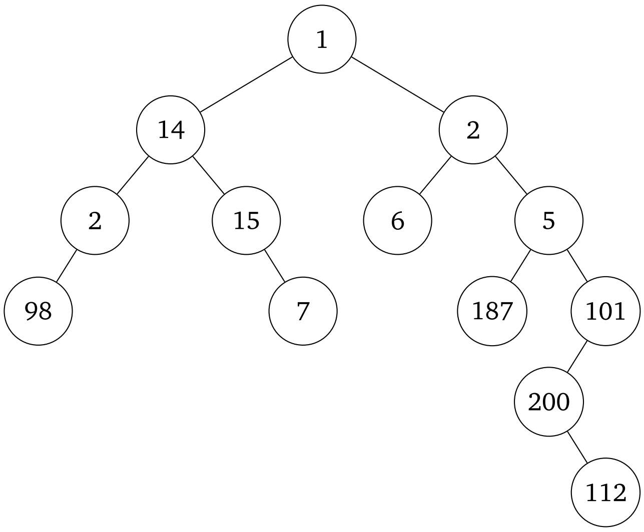
---

1. Look carefully at the following code for quick sort and fix the bugs.

```
import random

def qsort(l, fst, lst):
    if fst >= lst: return

    i, j = fst, lst
    pivot = l[random.randint(fst, lst)]

    while i <= j:
        while l[i] < pivot: i -= 1
        while l[j] > pivot: j += 1

        if i <= j:
            l[i], l[j] = l[j], l[i]
            i, j = j + 1, i - 1

    qsort(l, fst, j)
```

Score
/ 3

2. For the following tree, write down the order of nodes visited if we do (a) pre-order (b) in-order or (c) post-order traversal.



Score
/ 3

3. Write a recursive function to count the number of nodes in a Tree.

Score
/ 4

4. The height of a tree is the maximum number of levels in the tree. So, a tree with just one node has a height of 1. If the root has children which are leaves, the height of the tree is 2.

   The height of a TreeNode can be computed recursively using a simple algorithm: The height of a TreeNode with no children is 1. If it has children, the height is: *max of the height of its two sub-trees + 1*. (The tree in Question 2 has a height of 6 because it has two children: the left sub-tree has a height of 3 but the right sub-tree has a height of 5.)

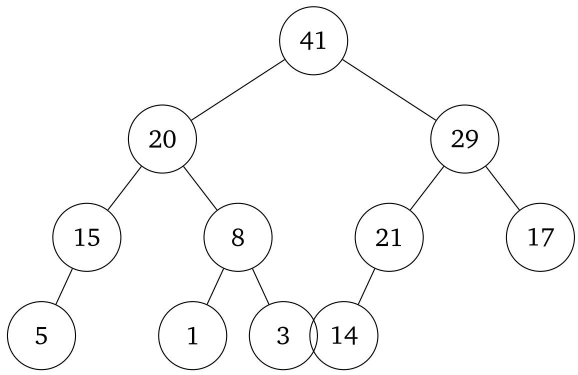
   Write a clean, recursive function for the TreeNode class that calculates the height based on the above statement.

Score
/ 4

5. Write a function which ensures that all nodes in a BST satisfy the BST property.

Score
/ 3

6. Look at the following max heap.



If we try to do heap sort, in the first iteration we will swap the root element with the last element and heapify the root again. What would the max heap look like after completing this *heapify* operation? (Not the complete heap sort. Just after this iteration.)

Score
/ 3